College of Computer Science and Information Technology
Department of Computer Engineering

ARTI 404 – Image Processing

Lab 5 Report: Spatial Filtering – Smoothing and Sharpening

Student Name Mohammed Omar Halawani

Student ID 2240001174

Academic Year 2025 – 2026

1. Objective

This lab applies spatial filters to an image: smoothing with box and Gaussian filters, and sharpening with unsharp masking and the Laplacian. All of them were implemented in Python with OpenCV, NumPy and Matplotlib.

2. Image Used

The lab refers to `Parrot.png`, which was not provided, so the Chelsea image is used for every task instead. It is saved as `lab5/chelsea.png`:

- **chelsea.png** – 451 × 300 RGB image of a cat, with fine detail in the fur and whiskers and a blurred background.

3. Setup

OpenCV loads images in BGR order, so the image is converted to RGB before plotting. For filtering, it is also converted to `float32` in the range [0, 1].

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Parrot.png is not provided, so use the included Chelsea image.
image = cv2.imread("chelsea.png")

if image is None:
    raise FileNotFoundError("Image could not be loaded.")

image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

4. Task 1 – Unsharp Masking

The image is blurred with a Gaussian of σ = 2. Subtracting the blurred image from the original leaves a mask that holds only the fine detail. That mask is multiplied by `amount = 1.5` and added back to the original, and the result is clipped to [0, 1]:

$$sharpened = f + 1.5 \cdot (f - G_{\sigma=2} * f)$$

```
# Gaussian sigma and sharpening amount
sigma = 2
amount = 1.5

# Convert image to float32 for processing
image_float = image.astype(np.float32) / 255.0

# Apply Gaussian blur
blurred = cv2.GaussianBlur(image_float, (0, 0), sigmaX=sigma, sigmaY=sigma)

# Perform Unsharp Masking
sharpened = np.clip(image_float + amount * (image_float - blurred), 0, 1)

# Plot original, blurred, and sharpened images
fig, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].imshow(image)
ax[0].set_title("Original Image")
ax[0].axis("off")
ax[1].imshow(blurred)
ax[1].set_title("Blurred Image")
ax[1].axis("off")
ax[2].imshow(sharpened)
ax[2].set_title("Sharpened (Unsharp Masking)")
ax[2].axis("off")
plt.tight_layout()
plt.show()
```

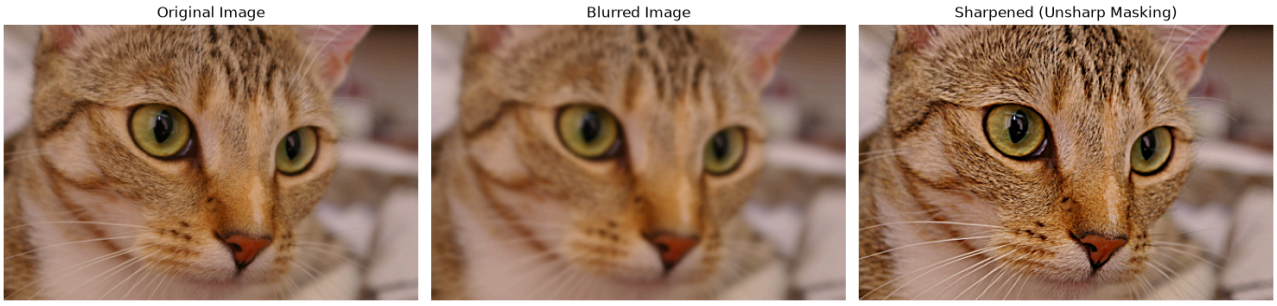


Figure 1: Original image, Gaussian-blurred image (σ = 2) and the unsharp-masked result (amount = 1.5).

Discussion: The blurred image loses the whiskers and the fur texture, so the difference between it and the original contains exactly those details. After adding it back with a gain of 1.5, the fur, whiskers and the edges of the eyes look much crisper than in the original. The variance of the Laplacian, a common sharpness measure, rises from about 399 for the original to about 2,411 for the sharpened image. The cost is a thin light or dark halo along strong edges, such as the rims of the eyes and the nose, and about 1.3% of the pixel values are clipped to 0 in dark regions. A smaller amount or a smaller σ would reduce these halos.

5. Assessment Task 1 – 7×7 Box Filter

A 7×7 kernel of ones is divided by 49 so that its weights sum to 1, and the image is convolved with it using `cv2.filter2D`. Each output pixel becomes the plain average of its 49 neighbours. Because the

kernel is normalized, the overall brightness of the image does not change.

```
# Convolve the image with a normalized 7x7 box filter
box_filter = np.ones((7, 7), dtype=np.float32) / 49.0
box_smoothed = cv2.filter2D(image_float, -1, box_filter)

fig, ax = plt.subplots(1, 2, figsize=(10, 5))
ax[0].imshow(image)
ax[0].set_title("Original Image")
ax[0].axis("off")
ax[1].imshow(box_smoothed)
ax[1].set_title("7x7 Box Filter")
ax[1].axis("off")
plt.tight_layout()
plt.show()
```

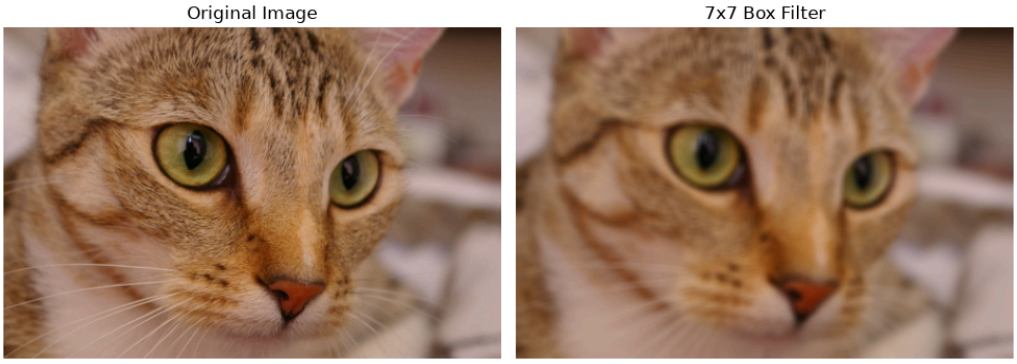


Figure 2: Original image and the result of the normalized 7×7 box filter.

Discussion: The box filter removes nearly all fine detail. The whiskers almost disappear, the fur becomes a smooth texture, and the Laplacian variance drops from about 399 to about 5. Every neighbour has the same weight, so a sharp edge is smeared evenly across the full 7-pixel width, and the edges of the eyes and nose look soft and slightly blocky rather than naturally out of focus.

6. Assessment Task 2 – 5×5 and 21×21 Gaussian Filters

The image is smoothed with Gaussian kernels of size 5×5 and 21×21. Passing σ = 0 lets OpenCV derive σ from the kernel size, using σ = 0.3·((k − 1)/2 − 1) + 0.8. This gives σ ≈ 1.1 for the 5×5 kernel, whose 1-D weights are [1, 4, 6, 4, 1] / 16, and σ ≈ 3.5 for the 21×21 kernel.

```
# Apply 5x5 and 21x21 Gaussian filters
gaussian_5 = cv2.GaussianBlur(image_float, (5, 5), 0)
gaussian_21 = cv2.GaussianBlur(image_float, (21, 21), 0)

fig, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].imshow(image)
ax[0].set_title("Original Image")
ax[0].axis("off")
ax[1].imshow(gaussian_5)
ax[1].set_title("5x5 Gaussian Filter")
ax[1].axis("off")
ax[2].imshow(gaussian_21)
ax[2].set_title("21x21 Gaussian Filter")
ax[2].axis("off")
plt.tight_layout()
plt.show()
```



Figure 3: Original image and the results of the 5×5 and 21×21 Gaussian filters.

Discussion: The 5×5 Gaussian gives a light blur. The fur texture is softened, but the whiskers and the outlines of the eyes are still visible (Laplacian variance ≈ 20). The 21×21 Gaussian gives a much stronger blur (≈ 2.3): the whiskers are gone and only the large shapes such as the eyes, nose and ears remain. The Gaussian gives the most weight to the centre pixel and less to pixels farther away, so the blur looks smoother and more natural than the box filter, without its blocky look. The 5×5 Gaussian keeps more detail than the 7×7 box filter, while the 21×21 Gaussian blurs more strongly than the box filter.

7. Assessment Task 3 – 3×3 Laplacian Sharpening

The image is converted to grayscale and float, and its Laplacian is computed with `cv2.Laplacian` and `ksize=3`. The Laplacian is a second-derivative filter: it is zero in flat regions and large near edges. Its centre coefficient is negative, so it is subtracted from the image to sharpen it: $g = f - \nabla^2 f$. The Laplacian has both positive and negative values, so for display it is scaled by 3 and shifted to mid-gray.

```
# Convert to grayscale and float32 before applying the 3x3 Laplacian
gray_image = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
gray_float = gray_image.astype(np.float32) / 255.0

laplacian = cv2.Laplacian(gray_float, cv2.CV_32F, ksize=3)
laplacian_sharpened = np.clip(gray_float - laplacian, 0, 1)

# Scale the signed Laplacian around mid-gray so its edges are visible.
laplacian_display = np.clip(0.5 + laplacian * 3, 0, 1)

fig, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].imshow(gray_image, cmap="gray", vmin=0, vmax=255)
ax[0].set_title("Original Grayscale Image")
ax[0].axis("off")
ax[1].imshow(laplacian_display, cmap="gray", vmin=0, vmax=1)
ax[1].set_title("3x3 Laplacian (Scaled for Display)")
ax[1].axis("off")
ax[2].imshow(laplacian_sharpened, cmap="gray", vmin=0, vmax=1)
ax[2].set_title("Laplacian Sharpened Image")
ax[2].axis("off")
plt.tight_layout()
plt.show()
```

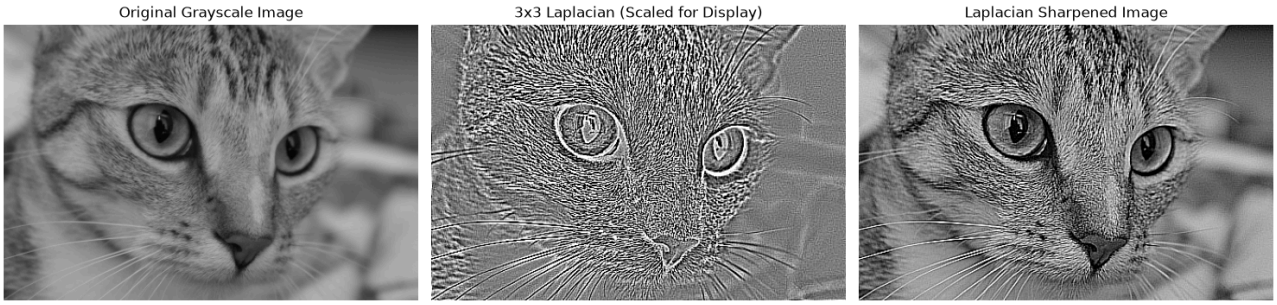


Figure 4: Grayscale image, its 3×3 Laplacian (scaled around mid-gray) and the Laplacian-sharpened image.

Discussion: In the Laplacian image, flat areas such as the blurred background are mid-gray, while the whiskers, the fur and the rims of the eyes show up as bright and dark lines. Subtracting it from the image strengthens exactly these edges, so the sharpened image shows clearly defined whiskers and fur. The Laplacian also amplifies noise and very fine texture, and the effect is much stronger than unsharp masking: about 8% of the pixel values are clipped to black or white, and the fur looks grainy. A smaller weight on the Laplacian, or smoothing the image first, would give a more natural result.

8. Conclusion

Smoothing filters average neighbouring pixels and remove fine detail, while sharpening filters do the opposite and strengthen it. The box filter is simple but smears edges evenly and looks blocky. The Gaussian filter produces a more natural blur, and its strength is controlled by the kernel size and σ. For sharpening, unsharp masking gave the most controllable and natural-looking result, because its strength can be adjusted with σ and the amount. The Laplacian sharpens edges strongly but also amplifies noise.

9. References

- Sandipan Dey, *Hands-On Image Processing with Python*, Packt.
- OpenCV: Smoothing Images – docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html
- OpenCV: Image Gradients (Laplacian) – docs.opencv.org/4.x/d5/d0f/tutorial_py_gradients.html